COMP 2233: Data Structures

Lecture Notes — Week 3: Abstract Data Types, Single Dimension Arrays, and Dynamic Vectors

Course Code & Title: COMP 2233 Data Structures
Instructor: Assoc. Prof. Dr. Ömer ÇETİN
Department: Computer Engineering
Academic Year: 2026 – 2027

Table of Contents

The Table of Contents below uses Word field codes and will populate automatically when opened in Microsoft Word or Google Docs.

1. Abstract Data Types & C Language Foundations

1.1 What is a Programming Language?

A **programming language** is a formalized, structured system of rules, notation, and symbols used to communicate instructions to a computer system. It bridges human problem-solving logic with physical computer hardware. Every programming language is governed by two complementary pillars:

- **Syntax:** The exact grammar, vocabulary, structural rules, and symbol arrangements required to form valid statements (e.g., semicolons, braces, keywords).
- **Semantics:** The underlying mathematical meaning, execution logic, and operational outcome of valid syntactical statements.

1.2 Low-Level vs. High-Level Programming Languages

Programming languages are categorized along an abstraction spectrum relative to physical computer architecture:

- **Low-Level Languages (Machine & Assembly):** Directly bound to CPU architecture registers and memory instructions. Machine code consists of raw binary digits (0s and 1s), while Assembly uses mnemonic opcodes (e.g., MOV, ADD). They offer maximum execution velocity and hardware control but lack portability.
- **High-Level Languages (C, C++, Java, Python):** Provide machine-independent abstractions, mathematical expressions, control structures (loops, conditionals), and custom data types. High-level code must be translated into machine instructions before execution.

1.3 Programming Language vs. Machine Language: The Compilation Pipeline

Computers natively execute only binary machine instructions. Transforming human-readable C source code into executable binary instructions follows a strict four-stage compilation pipeline:

Pipeline Stage	Input → Output	Primary Responsibility
1. Preprocessor	Source Code (.c) → Expanded Code (.i)	Expands macros (#define), pastes header contents (#include), strips comments.
2. Compiler	Expanded Code (.i) → Assembly Code (.s)	Translates C constructs into CPU-specific assembly opcodes; checks syntax.
3. Assembler	Assembly Code (.s) → Object File (.o/.obj)	Converts assembly mnemonics into raw relocatable machine binary code.
4. Linker	Object Files + Libraries → Executable (.exe)	Resolves function call addresses across modules and standard library implementations.

1.4 What is an IDE and Its Core Components?

An Integrated Development Environment (IDE) consolidates software development tools into a single graphical workspace. Core components include:

- **Source Code Editor:** Provides syntax highlighting, auto-completion (IntelliSense), indentation, and real-time error checking.
- **Compiler & Build Integration:** Automates toolchain invocations (GCC/Clang) via build configurations (Makefiles, CMake).
- **Interactive Debugger:** Enables line-by-line step execution (GDB/LLDB), breakpoint management, call stack inspection, and memory variable tracking.

1.5 Why C Language & RAM Memory Architecture

C remains the foundational language for data structures, systems programming, database engines, and OS kernels (Linux, Windows) because it combines high-level structured control with direct RAM physical address access.

> 📌 **RAM Memory Layout of a C Program**
>
> When a C executable is loaded into system memory, its address space is divided into 5 distinct segments:
>
> 1. Text Segment: Executable machine code instructions (Read-Only).
> 2. Data Segment: Global & static variables initialized with explicit non-zero values.
> 3. BSS Segment: Uninitialized global & static variables (zero-initialized by kernel).
> 4. Heap Area: Dynamically allocated memory managed at runtime via malloc()/free() or new/delete.
> 5. Stack Area: Automatic local function call frames, parameters, return addresses, and local variables.

1.6 Structured Programming & VSCode Environment Setup

Structured programming emphasizes modularity, sequence control, selection (if/switch), and iteration (for/while) without unstructured jump instructions (goto). In VSCode, C development requires the C/C++ extension pack and GCC toolchain configured via tasks.json (build script) and launch.json (GDB debug runner).

Source Code: First C Program Anatomy (main.c)

```
#include <stdio.h>  // Standard Input/Output Preprocessor Header

// Program entry point
int main(void) {
    // Formatted output to terminal
    printf("Welcome to COMP 2233 Data Structures!\n");
    return 0; // Signal successful exit status to operating system
}
```

1.7 What is an Abstract Data Type (ADT) & C Data Types

An Abstract Data Type (ADT) is a high-level mathematical specification of a data model that defines:

- **Public Interface:** The set of supported operations, preconditions, and postconditions available to client applications.
- **Implementation Abstraction Barrier:** Hides internal physical storage details (e.g., whether a Queue is backed by an Array or Linked List) from client code.

Primitive C data types provide the concrete building blocks for physical ADT implementations:

Type Keyword	Typical Size	Value Range	Format Specifier
char	1 Byte (8 bits)	-128 to 127 or 0 to 255	%c (character), %d (ASCII)
int	4 Bytes (32 bits)	-2,147,483,648 to 2,147,483,647	%d or %i
float	4 Bytes (32 bits)	1.2E-38 to 3.4E+38 (6 decimals)	%f
double	8 Bytes (64 bits)	2.3E-308 to 1.7E+308 (15 decimals)	%lf

1.8 Identifiers, Variables, Constants & Assignment Semantics

Identifiers are user-defined names assigned to variables, functions, and structs. Rules: must begin with a letter or underscore, cannot match reserved keywords, and are case-sensitive.

- **Variables:** Named RAM memory locations whose stored value can mutate during runtime execution.
- **Constants:** Immutable values defined via const qualifier (type-checked: const double PI = 3.14159;) or preprocessor directive (#define MAX_CAPACITY 100).
- **L-Value vs. R-Value Semantics:** An L-Value refers to a persistent memory locator that can appear on the left side of assignment (=). An R-Value is a temporary data value evaluated on the right side.

1.9 String Handling, Formatted I/O, Type Cast & Structures

C has no native string object. Text strings are stored as character arrays terminated by a mandatory null character ('\0'). Escape sequences like \n (newline) and \t (tab) format terminal output.

- **Formatted I/O:** printf() formats output streams. scanf() reads standard input; scalar variables must be passed using the address-of operator (&var) so scanf() can mutate memory directly.
- **Type Casting:** Converts one data type to another. Implicit coercion occurs automatically (widening). Explicit casting (double)total / count forces exact floating-point division, avoiding integer truncation.
- **Structure Definitions (struct):** Groups heterogeneous data elements into a single composite record. Member access uses dot operator (.) for struct values and arrow operator (->) for struct pointers.

Source Code: Structure Definition & Explicit Type Cast Example

```
#include <stdio.h>
#include <string.h>

typedef struct {
    int student_id;
    char name[50];
    int total_credits;
    int total_points;
} Student;

int main(void) {
    Student s1 = {1001, "Alice Smith", 15, 54};
    Student *ptr = &s1;

    // Explicit cast converts integer division to double
    double gpa = (double)ptr->total_points / ptr->total_credits;

    printf("Student ID: %d | Name: %s\n", ptr->student_id, ptr->name);
    printf("GPA (Calculated): %.2f\n", gpa);
    return 0;
}
```

2. Data Structures and Data Models

2.1 Core Definitions: Data, Data Structure, and Data Model

Understanding system architecture requires distinguishing three fundamental concepts:

- **Data:** Unprocessed, raw atomic facts, numbers, or characters lacking contextual meaning on their own.
- **Data Model:** An abstract mathematical specification defining logical relationships, constraints, and valid operations among data elements.
- **Data Structure:** A concrete physical implementation of a data model in RAM memory, defining memory layout, node linkages, and algorithmic procedures for storage and retrieval.

2.2 Linear vs. Non-Linear Data Structures Taxonomy

Data structures are divided into two main architectural classifications:

Category	Member Structures	Structural Characteristics
Linear Structures	Arrays, Linked Lists, Stacks, Queues	Elements are arranged sequentially in 1D order. Each node has a unique predecessor and successor. Traversal is single-pass linear.
Non-Linear Structures	Trees, Binary Search Trees, Graphs, Hash Tables	Elements are connected hierarchically (parent-child) or in complex network graphs. Nodes can connect to multiple neighbors.

2.3 Importance & Selection Framework

As Niklaus Wirth famously formulated in his classic computer science axiom:

> 📌 **Wirth's Axiom of Computer Science**
> Programs = Algorithms + Data Structures
>
> An algorithm dictates procedural step logic, but the physical data structure determines operational velocity, memory consumption, and system scalability. Choosing an inappropriate structure degrades runtime from O(1) or O(log N) down to O(N^2), making large-scale data processing infeasible.

Selecting the optimal data structure requires evaluating four core decision criteria:

- **1. Access Pattern:** Does the application perform frequent random lookups by numeric index (Static Array) or sequential node traversals?

- **2. Dynamic Resizing Needs:** Is dataset volume fixed at compile time, or must it expand continuously during runtime execution?
- **3. Insertion/Deletion Frequency:** Are items frequently inserted or deleted in the middle of the dataset (Pointer Linked List) or appended at boundaries?
- **4. Memory & Hardware Constraints:** Is RAM fragmented, or is CPU cache locality (contiguous spatial prefetching) paramount for high-frequency trading loops?

3. Single Dimension Arrays in C

3.1 Array Definition & Physical Memory Architecture

A Single Dimension Array is a fixed-size, contiguous sequence of homogeneous data elements accessible via integer indices. Key architectural properties:

- **Contiguous RAM Allocation:** Array elements occupy an unbroken, consecutive block of physical RAM addresses.
- **Zero-Based Indexing:** Indices range from 0 to N-1, where N is the total array capacity.
- **O(1) Direct Address Calculation Formula:** Because memory is contiguous, the physical RAM location of any element arr[i] is computed instantly in 1 CPU instruction without looping:

> 📌 **Physical Memory Address Calculation Formula**
> Address(arr[i]) = Base_Address + (i * Element_Size_in_Bytes)
>
> Where:
> • Base_Address: RAM memory address of first element arr[0] (e.g., 0x1000)
> • i: Zero-based target index
> • Element_Size_in_Bytes: Memory footprint returned by sizeof(DataType) (e.g., sizeof(int) = 4 bytes)
>
> Example: For int arr[4] stored at Base_Address 0x1000, Address(arr[2]) = 0x1000 + (2 * 4) = 0x1008.

3.2 Trade-offs & Usage Guidance for Static Arrays

Key Advantages of Static Arrays	Key Limitations of Static Arrays
• O(1) Constant Time Lookup: Direct index calculation enables instant random element access. • Maximum CPU Cache Locality: Contiguous sequential arrangement triggers hardware spatial cache line prefetching. • Minimal Memory Overhead: Requires zero pointer link storage overhead.	• Fixed Capacity: Array size is bound at declaration and cannot grow or shrink dynamically. • Costly Insertion & Deletion: Modifying middle elements demands O(N) element shifting. • Contiguous Memory Requirement: Can fail allocation if available RAM is fragmented.

3.3 Comprehensive C Array Operations Implementation

Below are procedural implementations for all 9 fundamental array operations, complete with C source code and operational time complexity bounds.

Source Code: 1 & 2. Creation, Declaration & Traversing Array Elements (O(N))

```
#include <stdio.h>

void traverseArray(const int arr[], int size) {
    printf("Array Contents (%d elements): [ ", size);
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]); // Access each index sequentially
    }
    printf("]\n");
}
```

Source Code: 3. Insertion at Specific Index (Right-Shifting) (O(N))

```
// Shifts elements right to make room at target index 'pos'
int insertElement(int arr[], int size, int capacity, int pos, int val) {
    if (size >= capacity || pos < 0 || pos > size) {
        printf("Error: Capacity limit reached or invalid position!\n");
        return size;
    }
    // Shift items right from tail down to pos
    for (int i = size - 1; i >= pos; i--) {
        arr[i + 1] = arr[i];
    }
    arr[pos] = val; // Insert value in vacated slot
    return size + 1; // Return new size
}
```

Source Code: 4. Deletion from Specific Index (Left-Shifting) (O(N))

```
// Overwrites target index by shifting subsequent items leftward
int deleteElement(int arr[], int size, int pos) {
    if (pos < 0 || pos >= size) {
        printf("Error: Invalid position!\n");
        return size;
    }
    // Shift items left to close memory gap
    for (int i = pos; i < size - 1; i++) {
```

```
        arr[i] = arr[i + 1];
    }
    return size - 1; // Return updated size
}
```

Source Code: 5 & 6. Linear Search (O(N)) and Bubble Sort (O(N^2))

```
int linearSearch(const int arr[], int size, int key) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == key) return i; // Key found
    }
    return -1; // Key absent
}

void bubbleSort(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Source Code: 7, 8 & 9. Updating (O(1)), Reversing (O(N)), and Merging (O(N+M))

```
// Direct update in O(1) time
void updateElement(int arr[], int pos, int newVal) {
    arr[pos] = newVal;
}

// In-place two-pointer reversing
void reverseArray(int arr[], int size) {
    int left = 0, right = size - 1;
    while (left < right) {
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;
        left++; right--;
    }
}

// Merges array A (size n) and array B (size m) into C (size n+m)
void mergeArrays(const int A[], int n, const int B[], int m, int C[]) {
```

```
    for (int i = 0; i < n; i++) C[i] = A[i];
    for (int j = 0; j < m; j++) C[n + j] = B[j];
}
```

3.4 Practical Engineering Examples with Arrays

Source Code: Example: Encapsulating Array & Size in a C Struct (Array ADT)

```
#include <stdio.h>
#define CAPACITY 100

typedef struct {
    int data[CAPACITY];
    int size;
} ArrayADT;

void initArray(ArrayADT *arr) {
    arr->size = 0;
}

void appendArray(ArrayADT *arr, int val) {
    if (arr->size < CAPACITY) {
        arr->data[arr->size++] = val;
    }
}

int main(void) {
    ArrayADT list;
    initArray(&list);
    appendArray(&list, 10);
    appendArray(&list, 25);
    appendArray(&list, 40);

    printf("Array ADT size: %d | Elements: [ %d, %d, %d ]\n",
           list.size, list.data[0], list.data[1], list.data[2]);
    return 0;
}
```

4. Dynamic Vectors in C++ (STL)

4.1 What is a Vector? Concept & Declaration Patterns

A Vector (std::vector) is a dynamic sequence container provided by the C++ Standard Template Library (#include <vector>). Like static arrays, vectors store elements in contiguous RAM memory, offering O(1)

random access indexing. However, vectors automatically resize themselves when items are inserted or removed.

Source Code: C++ Vector Declaration and Initialization Patterns

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> empty_vec;              // Empty vector (size=0, capacity=0)
    std::vector<int> sized_vec(5);           // 5 elements initialized to 0
    std::vector<int> filled_vec(5, 100);     // 5 elements filled with 100
    std::vector<int> init_list = {10, 20, 30}; // Initializer list (C++11)

    std::cout << "Init List Size: " << init_list.size() << "\n";
    return 0;
}
```

4.2 Vector Operations & Amortized O(1) Buffer Doubling

Vectors distinguish between active element count (size) and total allocated RAM capacity (capacity):

- **size():** Returns the actual number of active elements stored.
- **capacity():** Returns the number of allocated memory slots available before re-allocation is required.
- **Amortized O(1) Memory Doubling Strategy:** When push_back() is invoked and size == capacity, the vector allocates a new memory block twice the capacity (2 * Capacity), copies existing elements across, and deallocates the old block. Because memory doubling occurs infrequently (at sizes 1, 2, 4, 8, 16, 32...), the amortized time complexity of push_back() remains O(1) constant time.

Source Code: Essential C++ STL Vector Operations Demo

```
#include <iostream>
#include <vector>
#include <stdexcept>

int main() {
    std::vector<int> vec;

    // 1. Appending elements (push_back)
    vec.push_back(10);
    vec.push_back(20);
    vec.push_back(30);

    // 2. Element access: [] vs .at()
    std::cout << "Unchecked access [0]: " << vec[0] << "\n";
    std::cout << "Checked access .at(1): " << vec.at(1) << "\n";

    // Exception-safe bounds check
```

```
    try {
        std::cout << vec.at(10); // Out of bounds!
    } catch (const std::out_of_range& e) {
        std::cout << "Caught Exception: " << e.what() << "\n";
    }

    // 3. Tail deletion (pop_back)
    vec.pop_back(); // Removes 30 in O(1) time

    // 4. Iterator-based insert and erase
    vec.insert(vec.begin() + 1, 15); // Inserts 15 at index 1 (O(N))
    vec.erase(vec.begin());          // Removes element at index 0 (O(N))

    std::cout << "Final Vector Size: " << vec.size()
              << " | Capacity: " << vec.capacity() << "\n";
    return 0;
}
```

4.3 Comparative Analysis: Static C Array vs. Dynamic C++ Vector

Understanding when to employ static C arrays versus dynamic C++ vectors is critical for system design:

Comparison Metric	Static C Array	Dynamic C++ Vector (std::vector)
Memory Allocation	Fixed at compile time / declaration.	Dynamic automatic runtime resizing.
Size Tracking	Manual integer variable tracking required.	Built-in .size() and .capacity() methods.
Memory Location	Local Stack area or Heap buffer.	Heap buffer encapsulated by Stack wrapper object.
Random Access Speed	O(1) constant time (fast direct lookup).	O(1) constant time (identical indexing speed).
Safety & Bounds Check	No bounds checking (risk of segfaults).	.at() method provides exception-safe checking.
Memory Cleanup	Manual free()/delete[] for heap arrays.	Automatic RAII destructor cleanup when out of scope.

Summary Guidance: Use static C arrays in hardware-constrained, embedded, or real-time environments where memory capacity is fixed and deterministic. Use C++ vectors as the default sequence container

for modern application engineering requiring flexible dynamic growth and automated resource management.